

Assignment 9

Team 27: Bhavdeep Arora, Parsa Kermani Pour, Arshia Sharifi

For this assignment, we utilized Oracle Instant Client, linked with Python, and styled the assignment using HTML.

We first begin by installing the instant client from the Oracle website

<https://www.oracle.com/ca-en/database/technologies/instant-client/winx64-64-downloads.html>

Then extract the zip file to:

- On MacOS: /Users/<username>/instantclient
- On Windows: C:\Users\<username>\instantclient

And add that location to your PATH.

The following snippet of code shows the implementation of how we connect Oracle to our host using InstantClient.



```
CPS510 > CPS510-Hospital-Pharmacy-DBMS > A9 > main.py
1  import os
2  import oracledb
3  from dotenv import load_dotenv
4
5  load_dotenv()
6
7  oracledb.init_oracle_client(lib_dir=os.getenv("ORACLE_CLIENT_LIB_DIR"))
8
9  conn = oracledb.connect(
10     user=os.environ.get("ORACLE_USERNAME"),
11     password=os.environ.get("ORACLE_PASSWORD"),
12     host="oracle.scs.ryerson.ca",
13     port=1521,
14     sid="orcl"
15 )
16
17 print("Connected:", conn.version)
18 conn.close()
```

We created a .env file where ORACLE_CLIENT_LIB_DIR is the address added to PATH before, and the rest have to be filled out personally.

```
CPS510 > CPS510-Hospital-Pharmacy-DBMS > A9 > ⚙ .env.template
```

```
1 ORACLE_HOST=  
2 ORACLE_PORT=  
3 ORACLE_SID=  
4 ORACLE_USERNAME=  
5 ORACLE_PASSWORD=  
6 ORACLE_CLIENT_LIB_DIR=  
7
```

We have also implemented Flask for a virtual environment for a more consistent approach to the application.

Looking at the code beforehand, we have the following main functions:

`execute_query()` opens a database connection, runs the given SQL using `cursor.execute()`, and fetches all results. It then returns the column names and rows in a dictionary before closing the connection.

```
def execute_query(sql):  
    conn = get_connection()  
    cursor = conn.cursor()  
    cursor.execute(sql)  
    columns = [desc[0] for desc in cursor.description]  
    rows = cursor.fetchall()  
    cursor.close()  
    conn.close()  
    return {'columns': columns, 'rows': rows}
```

`cursor.execute()` sends a SQL statement from Python to the Oracle database and tells the database to run it. After execution, you can fetch results (for SELECT queries) or commit changes (for INSERT/UPDATE/DELETE).

```

def execute_sql_file(file_path):
    conn = get_connection()
    cursor = conn.cursor()

    with open(file_path, 'r') as f:
        sql_content = f.read()

    lines = [line for line in sql_content.split('\n') if not line.strip().startswith('--')] and
    sql_content = '\n'.join(lines)
    statements, current, in_plsql = [], [], False

    for line in sql_content.split('\n'):
        upper = line.strip().upper()
        if 'CREATE OR REPLACE TRIGGER' in upper or 'CREATE TRIGGER' in upper:
            in_plsql = True
            current.append(line)
            continue
        if upper.startswith('BEGIN') and in_plsql:
            current.append(line)
            continue
        if line.strip() == '/' and in_plsql:
            in_plsql = False
            stmt = '\n'.join(current).strip()
            if stmt:
                statements.append(stmt)
            current = []
            continue
        current.append(line)

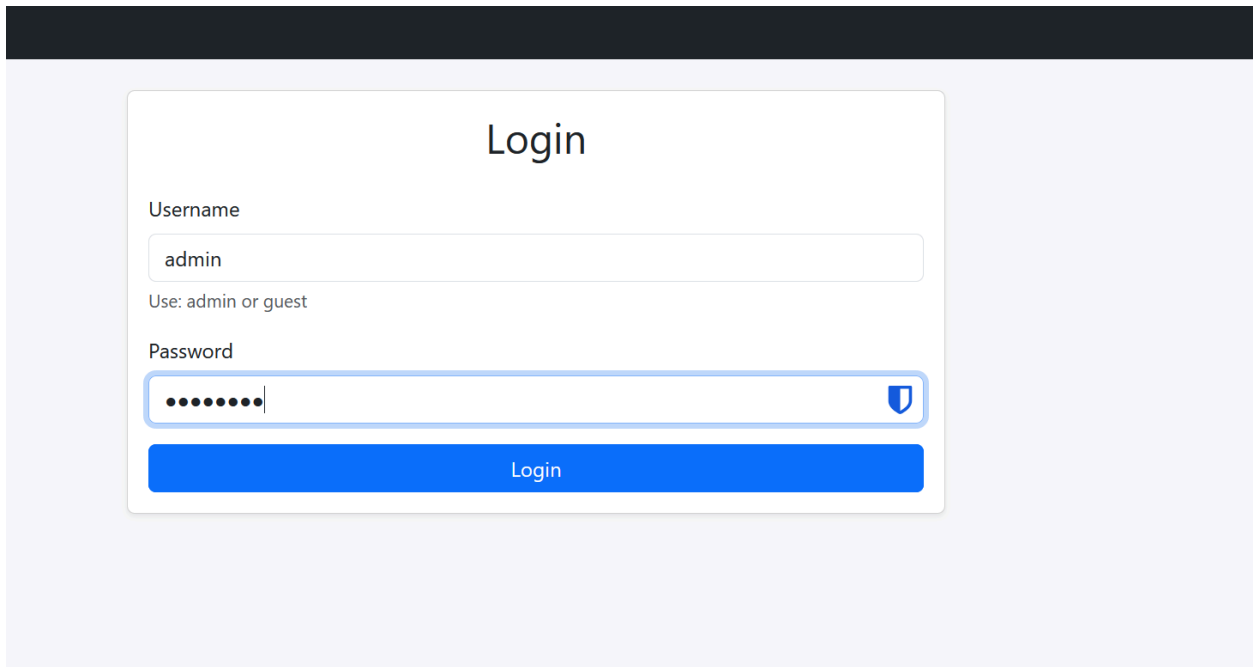
```

```

        if line.rstrip().endswith(';'):
            if not in_plsql:
                stmt = '\n'.join(current).strip().rstrip(';')
                if stmt:
                    statements.append(stmt)
                current = []
    errors, success = [], 0
    for stmt in statements:
        if stmt and stmt.strip():
            try:
                cursor.execute(stmt)
                success += 1
            except Exception as e:
                errors.append(f"{stmt[:80]}... | Error: {str(e)}")
    conn.commit()
    cursor.close()
    conn.close()
    return {'success': success, 'errors': errors}

```

You can then run the program by running `python app/app.py` in your assignment directory which takes you to <http://127.0.0.1:5000/>, which is ur localhost:5000 port. The following website

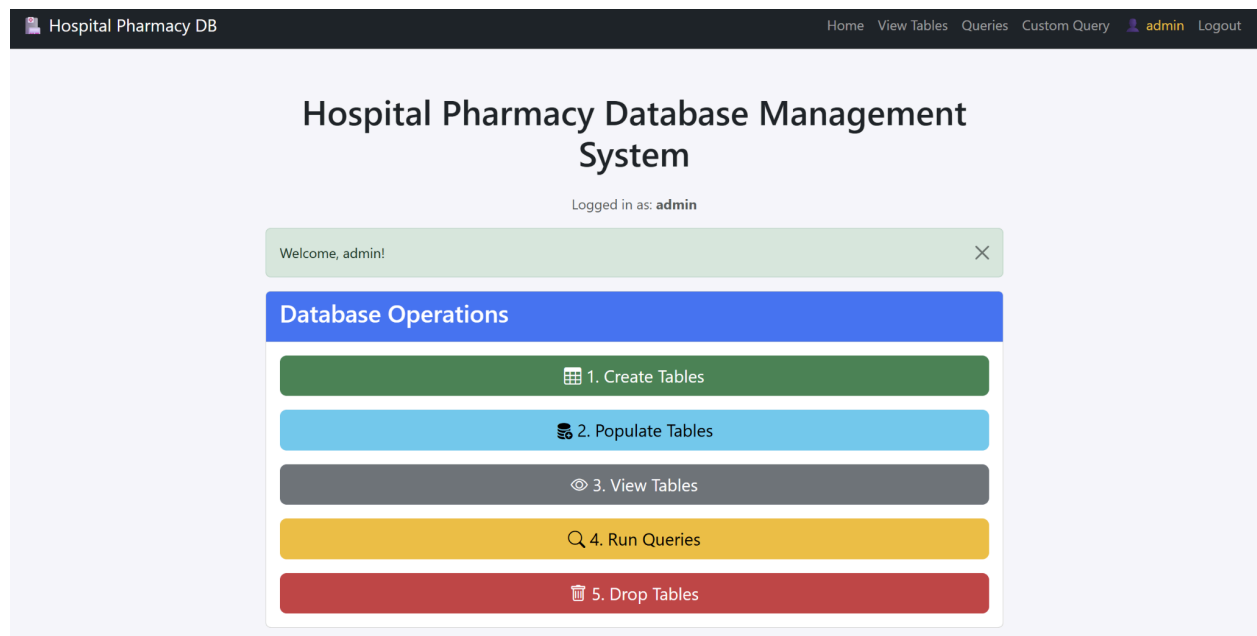


```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "").strip()

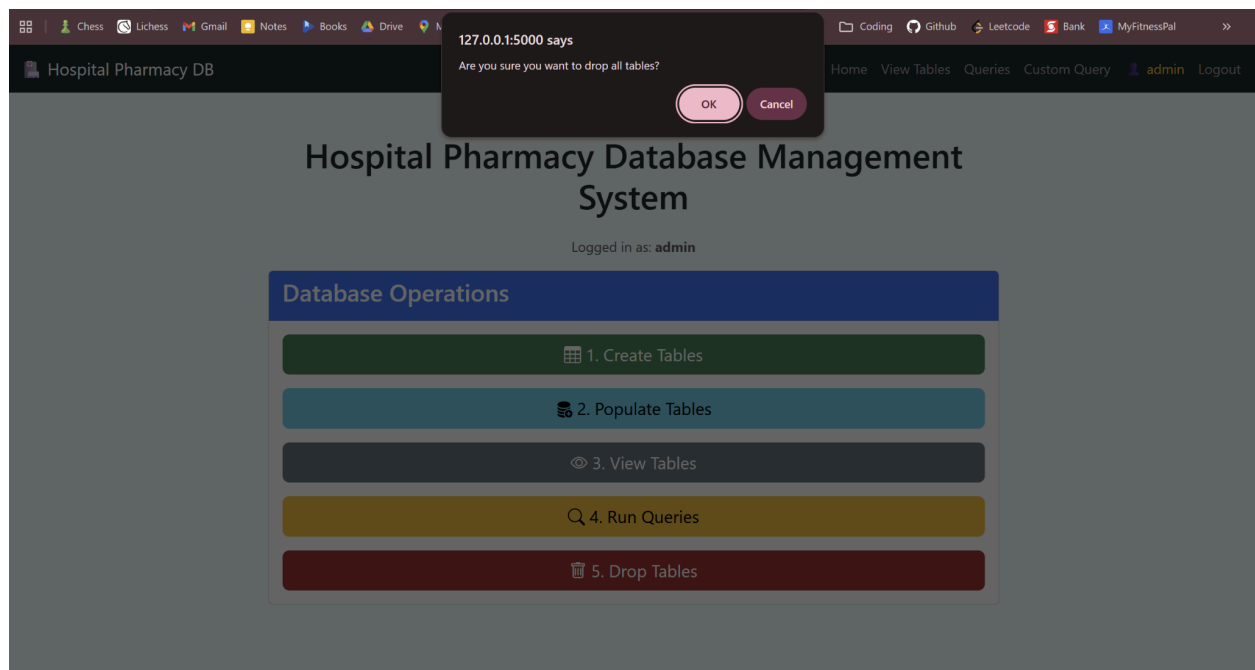
        if username in USERS and USERS[username] == password:
            session['username'] = username
            flash(f"Welcome, {username}!", "success")
            return redirect("/")
        else:
            flash("Invalid username or password", "danger")

    return render_template("login.html")
```

We have created two users, admin and guest, for this assignment, with the passwords being admin123 and guest123, respectively. After logging in, you should be able to see the following page:



We first drop the tables



Hospital Pharmacy Database Management System

Logged in as: **admin**

Tables dropped successfully!



Database Operations

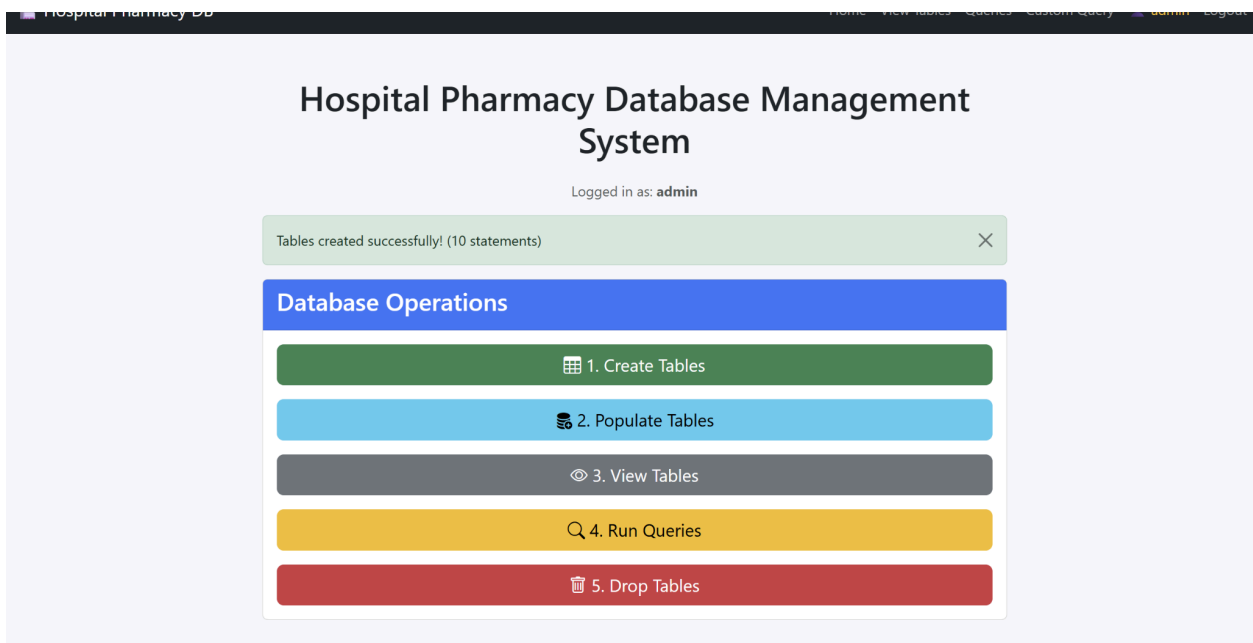
 1. Create Tables

 2. Populate Tables

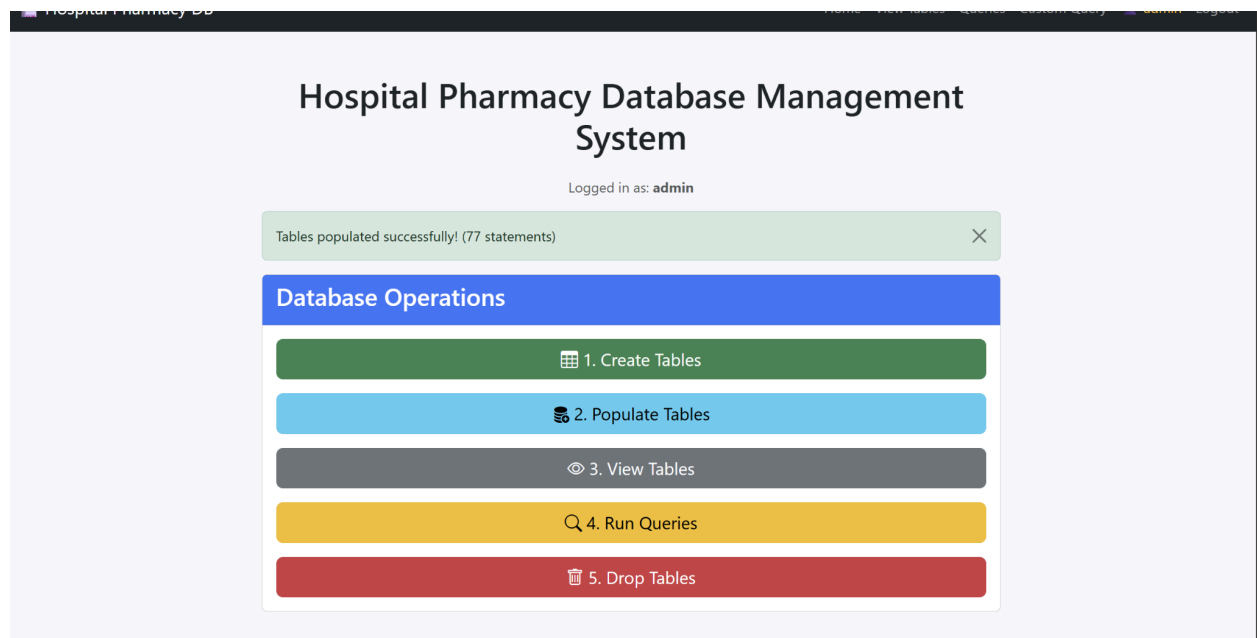
 3. View Tables

 4. Run Queries

 5. Drop Tables



And then once the tables have been dropped, we can create the tables. If tables already exist when creating tables, then it will do nothing and show an error popup.



We populate the table, i.e, add data to each table

 Hospital Pharmacy DB

HomeView TablesQueriesCustom QueryadminLogout

Database Tables

Available Tables

 DEPARTMENT
 DOCTOR
 PHARMACIST
 PATIENT
 SUPPLIER
 MEDICATION
 SUPPLIED_FROM
 PRESCRIPTION

[← Back to Home](#)

 Hospital Pharmacy DB

HomeView TablesQueriesCustom QueryadminLogout

Table: DEPARTMENT

DEPARTMENT_ID	DEPARTMENT_NAME
1	Cardiology
2	Neurology
3	Pediatrics

Total records: 3

[← Back to Tables](#)[Home](#)

These are some of the tables and what their data looks like.

Table: DOCTOR

DOCTOR_ID	FULL_NAME	EMAIL	DEPARTMENT_ID
123456789	Dr. Sarah Johnson	sarah.johnson@hospital.com	1
234567890	Dr. Michael Chen	michael.chen@hospital.com	2
345678901	Dr. Emily Rodriguez	emily.rodriguez@hospital.com	3
456789012	Dr. David Kim	david.kim@hospital.com	1
567890123	Dr. Lisa Thompson	lisa.thompson@hospital.com	2

Total records: 5

[← Back to Tables](#)[Home](#)

Table: SUPPLIER

SUPPLIER_ID	SUPPLIER_NAME	PHONE_NUMBER	EMAIL	DELIVERY_DURATION_DAYS	SUPPLIER_ADDRESS
1	PharmaTech Solutions	416-555-1001	orders@pharmatech.com	3	100 Industry Blvd, Toronto
2	MediCore Distribution	905-555-1002	sales@medicore.com	5	200 Commerce Way, Mississauga
3	VitalCare Supply Co	647-555-1003	info@vitalcare.com	2	300 Medical Ave, Brampton
4	CanadaRx Pharmaceuticals	416-555-1004	contact@canadarx.com	7	400 Science Park, Toronto
5	MedSupply Plus	905-555-1005	support@medsupplyplus.com	1	500 Quick Lane, Mississauga

Total records: 5

[← Back to Tables](#)[Home](#)

 Hospital Pharmacy DB

HomeView TablesQueriesCustom QueryadminLogout

Advanced Database Queries

Available Queries

Query 1: Doctors with more than 2 unique patients
Find all doctors who have prescribed to more than 2 different patients

Query 2: Patients with prescriptions from multiple suppliers
Patients who have received medications from at least 2 different suppliers

Query 3: Suppliers with unique medications
Medications that are only available from one specific supplier

Query 4: Departments with above-average prescriptions
Departments where doctors prescribe more than the hospital average

Query 5: Patients with multiple prescriptions
Find patients who have more than one prescription

← Back to Home

127.0.0.1:5000/query/2

You can also choose from predefined query selection

 Hospital Pharmacy DB

HomeView TablesQueriesCustom QueryadminLogout

Doctors with more than 2 unique patients

DOCTOR_NAME	UNIQUE_PATIENTS
Dr. David Kim	4
Dr. Sarah Johnson	3
Dr. Michael Chen	3
Dr. Lisa Thompson	3
Dr. Emily Rodriguez	3

Total records: 5

← Back to Queries

Home

You can also add a custom query. For our implementation, unlike SQL, we don't add `;` at the end.

 Hospital Pharmacy DB

HomeView TablesQueriesCustom QueryadminLogout

Execute Custom SQL Query

Enter Your SQL Query

Enter SQL query (e.g., SELECT * FROM patient WHERE dob > DATE '2000-01-01')



▶ Execute Query

🏠 Back to Home

 Hospital Pharmacy DB

HomeView TablesQueriesCustom QueryadminLogout

Execute Custom SQL Query

Enter Your SQL Query

SELECT * FROM prescription



▶ Execute Query

🏠 Back to Home

SELECT * FROM prescription

▶ Execute Query

🏠 Back to Home

Query Results

PRESCRIPTION_ID	NOTES	DATE_CREATED	DATE_DISPENSED	PATIENT_ID	DOCTOR_ID	PHARMACIST_ID	SUPPLIER_ID	DRUG_NAME
1	Take twice daily with food	2025-09-15 10:30:00	2025-09-16 11:15:00	629518374062	123456789	678901234	4	Amoxicillin
2	Once daily before breakfast	2025-09-18 14:15:00	2025-09-19 15:30:00	847392561048	456789012	789012345	2	Ibuprofen
3	As needed for chest pain	2025-09-20 09:45:00	2025-09-22 09:20:00	482917635841	234567890	890123456	3	Metformin
4	Liquid form, 5ml twice daily	2025-09-19 16:20:00	2025-09-20 14:45:00	629518374062	123456789	901234567	5	Insulin
9	After meals	2025-09-27 10:45:00	2025-09-27 12:00:00	391856402739	123456789	678901234	3	Ibuprofen
10	Before sleep	2025-09-28 11:15:00	NULL	847392561048	234567890	NULL	4	Ibuprofen
11	Morning dose	2025-09-29 12:00:00	2025-09-29 13:00:00	482917635841	456789012	789012345	2	Aspirin
12	Twice daily	2025-09-30 13:30:00	NULL	629518374062	345678901	NULL	1	Metformin
13	As needed for fever	2025-10-01 14:45:00	2025-10-01 15:00:00	756284193057	567890123	890123456	2	Acetaminophen
14	Before meals	2025-10-02 15:20:00	NULL	482917635841	123456789	NULL	5	Omeprazole
15	Night dose	2025-10-03 16:10:00	NULL	391856402739	234567890	NULL	3	Atorvastatin
16	Take with food	2025-10-04 17:00:00	NULL	629518374062	456789012	NULL	4	Amoxicillin
17	Once daily	2025-10-05 18:30:00	NULL	847392561048	345678901	NULL	2	Lisinopril
18	After lunch	2025-10-06 19:45:00	NULL	391856402739	567890123	NULL	1	Aspirin

Total records: 18